PYTHON FOR BEGINNERS · SESSION 8

Tuples and Sets

Data that cannot change and data without duplicates: tuples, immutability, unpacking and set operations.

Recap

Ask the class:

- What does `nums[::-1]` do to a list?
- What is the difference between `append()` and `extend()`?
- Which list method **returns None**?

What is a tuple?

A tuple is an **ordered collection** inside round brackets `( )`. It looks like a list, but it **cannot be changed** after it is made.

tuples.py

```
point = (3, 4)
colors = ("red", "green", "blue")
mixed = (1, "two", 3.0)
empty = ()
print(point, colors, mixed, empty)
print(len(colors))
print(type(point))
```

OUTPUT

```
(3, 4) ('red', 'green', 'blue') (1, 'two', 3.0) ()
3
<class 'tuple'>
```

- **Ordered:** every item has an index, starting from 0.
- **Immutable:** items cannot be changed, added or removed.
- **Mixed types allowed** and **duplicates allowed**, like a list.

The one-item tuple

The **comma** makes the tuple, not the brackets. A single item needs a **trailing comma**.

```
a = (5)
b = (5,)
c = 5,
print(type(a), type(b), type(c))
```

OUTPUT

```
<class 'int'> <class 'tuple'> <class 'tuple'>
```

Making a tuple from other things

```
print(tuple("abc"))
print(tuple([1, 2, 3]))
print(tuple(range(3)))
```

OUTPUT

```
('a', 'b', 'c')
(1, 2, 3)
(0, 1, 2)
```

Ask

What type is `(5)`? Why is `(5,)` different?

Indexing, slicing and methods

Reading a tuple works **exactly like a list or a string**: indexes, negative indexes and slices all work.

```
days = ("Mon", "Tue", "Wed", "Thu", "Fri")
print(days[0], days[-1])
print(days[1:3])
print(days[::-1])
print("Tue" in days)
```

OUTPUT

```
Mon Fri
('Tue', 'Wed')
('Fri', 'Thu', 'Wed', 'Tue', 'Mon')
True
```

Only two methods

Because a tuple cannot change, it has only **two methods**, and both just read.

METHOD	WHAT IT DOES
`count(x)`	how many times `x` appears
`index(x)`	position of the **first** `x`

```
t = (1, 2, 2, 3, 2)
print(t.count(2))
print(t.index(3))
```

OUTPUT

```
3
3
```

Immutability

Immutable means the tuple cannot be changed after it is created. Any try to change it gives an **error**.

Try it: read the error

```
t = (10, 20, 30)
t[0] = 99
```

ERROR

```
  File "your code", line 2, in <module>
TypeError: 'tuple' object does not support item assignment
```

Try it: no append either

```
t = (10, 20, 30)
t.append(40)
```

ERROR

```
  File "your code", line 2, in <module>
AttributeError: 'tuple' object has no attribute 'append'
```

How to "change" a tuple

You cannot change it, but you can **build a new tuple** and give it the same name.

```python
t = (10, 20, 30)
t = t + (40,)
print(t)
t = t[:2] + (99,) + t[3:]
print(t)
```

OUTPUT

```
(10, 20, 30, 40)
(10, 20, 99, 40)
```

Strings are immutable too

```python
s = "Python"
print(s.upper())
print(s)
nums = [1, 2, 3]
nums[0] = 100
print(nums)
```

OUTPUT

```
PYTHON
Python
[100, 2, 3]
```

Careful

A tuple can hold a **list**, and that list can still change. The tuple only guarantees that it keeps the **same objects**.

```python
t = (1, [2, 3])
t[1].append(4)
print(t)
```

OUTPUT

```
(1, [2, 3, 4])
```

Tuple or list?

Use a **list** when the data will change. Use a **tuple** when the data should stay fixed.

	LIST	TUPLE
Brackets	`[ ]`	`( )`
Can be changed	**Yes**	**No**
Methods	many	only `count`, `index`
Good for	a shopping list, marks that grow	a point `(x, y)`, a date, fixed settings

- **Safe:** nobody can change the data by mistake.
- **Slightly faster** and smaller than a list.
- Tuples can be **dictionary keys**, lists cannot (next class).
- A function that **returns many values** returns a tuple.

Ask

Would you store the days of the week in a list or a tuple? Why?

Unpacking

Unpacking copies the items of a tuple into separate variables in one line. The number of variables must **match** the number of items.

```
point = (3, 4)
x, y = point
print(x, y)

name, age, city = ("Aman", 20, "Delhi")
print(name, age, city)
```

OUTPUT

```
3 4
Aman 20 Delhi
```

Unpacking works with **lists and strings** too, and the brackets can be left out.

Try it: wrong number of variables

```
a, b = (1, 2, 3)
```

ERROR

```
  File "your code", line 1, in <module>
ValueError: too many values to unpack (expected 2)
```

Swapping two variables

```
a, b = 1, 2
a, b = b, a
print(a, b)
```

OUTPUT

```
2 1
```

Ask

In most languages swapping needs a third variable. Why does Python not?

Unpacking with *

A variable with a **star** `*` collects all the **extra items** into a **list**.

```
first, *rest = (1, 2, 3, 4)
print(first, rest)

*start, last = (1, 2, 3, 4)
print(start, last)

a, *mid, z = (1, 2, 3, 4, 5)
print(a, mid, z)
```

OUTPUT

```
1 [2, 3, 4]
[1, 2, 3] 4
1 [2, 3, 4] 5
```

- Only **one** starred variable is allowed in one unpacking.
- The starred variable is always a **list**, even if it gets nothing (`[]`).

Unpacking in loops

```
pairs = [(1, "one"), (2, "two"), (3, "three")]
for n, word in pairs:
    print(n, word)

for i, ch in enumerate("abc"):
    print(i, ch)
```

```
OUTPUT
1 one
2 two
3 three
0 a
1 b
2 c
```

Returning many values

```
def min_max(nums):
    return min(nums), max(nums)

result = min_max([4, 9, 2, 7])
print(result, type(result))

low, high = min_max([4, 9, 2, 7])
print(low, high)
```

```
OUTPUT
(2, 9) <class 'tuple'>
2 9
```

Tuple operations

Operators **build new tuples**; the old ones stay the same.

```
a = (1, 2)
b = (3, 4)
print(a + b)
print(a * 3)
print(max(a + b), min(a + b), sum(a + b))
print((1, 2) < (1, 3))
```

OUTPUT

```
(1, 2, 3, 4)
(1, 2, 1, 2, 1, 2)
4 1 10
True
```

Looping over a tuple

```
colors = ("red", "green", "blue")
for c in colors:
    print(c)

for i in range(len(colors)):
    print(i, colors[i])
```

OUTPUT

```
red
green
blue
0 red
1 green
2 blue
```

Tuple to list and back

To change a tuple the long way: **convert to a list**, change it, **convert back**.

```
t = (3, 1, 2)
items = list(t)
items.sort()
t = tuple(items)
print(t)
print(sorted((3, 1, 2)))
```

OUTPUT

```
(1, 2, 3)
[1, 2, 3]
```

Note

`sorted()` always gives a **list**, even for a tuple.

What is a set?

A set is a collection of **unique items** inside curly brackets `{ }`. Duplicates are **removed automatically**, and there is **no order**.

sets.py

```
s = {3, 1, 2, 3, 1}
print(s)
print(len(s))
print(type(s))
```

OUTPUT

```
{1, 2, 3}
3
<class 'set'>
```

- **Unique:** every value appears only once.
- **Unordered:** no index, so `s[0]` is an error.
- **Mutable:** you can add and remove items.
- Items must be **immutable**: numbers, strings and tuples are fine, lists are not.

Making a set

```
print(sorted(set("hello")))
print(list(set([1, 2, 2, 3, 3, 3])))

print(type({}), type(set()))
```

OUTPUT

```
['e', 'h', 'l', 'o']
[1, 2, 3]
<class 'dict'> <class 'set'>
```

Careful

`{}` is an empty **dictionary**, not an empty set. Use **`set()`** for an empty set.

Using a set

There are no indexes, but you can **check membership** and **loop**. Checking `in` on a set is very **fast**.

```
s = {1, 2, 3}
print(2 in s, 9 in s)
print(len(s))
for n in s:
    print(n)
```

OUTPUT

```
True False
3
1
2
3
```

What can go inside a set

Try it: a list cannot go inside a set

```
s = {[1, 2], 3}
```

ERROR

```
  File "your code", line 1, in <module>
TypeError: unhashable type: 'list'
```

Try it: a tuple can

```
s = {(1, 2), 3}
print(len(s))
```

OUTPUT

```
2
```

Removing duplicates from a list

```
nums = [1, 2, 2, 3, 4, 4, 5]
print(sorted(set(nums)))
```

OUTPUT

```
[1, 2, 3, 4, 5]
```

Careful

A set has **no order**, so use `sorted()` when you need the items in order.

Set methods

Set methods **change the set itself**, like list methods.

METHOD	WHAT IT DOES
`add(x)`	adds one item (nothing happens if it is already there)
`update(items)`	adds **many** items
`remove(x)`	removes `x`, **error** if missing
`discard(x)`	removes `x`, **no error** if missing
`pop()`	removes and returns **any** item
`clear()`	removes everything

```
s = {1, 2}
s.add(3)
s.add(2)
print(s)
s.update([3, 4, 5])
print(s)
s.remove(5)
s.discard(99)
print(s)
```

OUTPUT

```
{1, 2, 3}
{1, 2, 3, 4, 5}
{1, 2, 3, 4}
```

Try it: remove vs discard

```
s = {1, 2, 3}
s.discard(99)
print("discard is quiet")
s.remove(99)
```

ERROR

```
discard is quiet
  File "your code", line 4, in <module>
KeyError: 99
```

Ask

Which one would you use when you are **not sure** the item is in the set?

Set operations

Sets follow **school maths**. Every operation has an **operator** and a **method**, and gives a **new set**.

OPERATION	OPERATOR	METHOD	GIVES
Union	`a | b`	`a.union(b)`	everything in **either**
Intersection	`a & b`	`a.intersection(b)`	only what is in **both**
Difference	`a - b`	`a.difference(b)`	in `a` but **not** in `b`
Symmetric difference	`a ^ b`	`a.symmetric_difference(b)`	in **one but not both**

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)
print(a & b)
print(a - b)
print(b - a)
print(a ^ b)
```

OUTPUT

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{5, 6}
{1, 2, 5, 6}
```

Same thing with methods

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
print(a.symmetric_difference(b))
```

OUTPUT

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```

Careful

`a - b` and `b - a` are **different**. The order matters for difference, not for union or intersection.

Comparing sets

Sets can be compared as **subsets** and **supersets**.

CHECK	OPERATOR	METHOD
`x` is inside `y`	`x <= y`	`x.issubset(y)`
`y` contains `x`	`y >= x`	`y.issuperset(x)`
nothing in common		`x.isdisjoint(y)`

```
x = {1, 2}
y = {1, 2, 3}
print(x <= y, x.issubset(y))
print(y >= x, y.issuperset(x))
print({1, 2}.isdisjoint({3, 4}))
print({1, 2}.isdisjoint({2, 3}))
print({1, 2, 3} == {3, 2, 1})
```

OUTPUT

```
True True
True True
True
False
True
```

Updating in place

Each operator has an **in-place** form that changes the set: `|=`, `&=`, `-=`, `^=`.

```
a = {1, 2, 3}
a |= {4}
print(a)
a &= {1, 2, 4, 9}
print(a)
a -= {1}
print(a)
a ^= {2, 7}
print(a)
```

OUTPUT

```
{1, 2, 3, 4}
{1, 2, 4}
{2, 4}
{4, 7}
```

frozenset

A **frozenset** is an **immutable set**. It cannot change, so it can go inside another set.

```
fs = frozenset([1, 2, 3])
print(fs)
fs.add(4)
```

ERROR

```
frozenset({1, 2, 3})
  File "your code", line 3, in <module>
AttributeError: 'frozenset' object has no attribute 'add'
```

Sets in real life

Two classes, and we want to know who is in **both**, in **either**, or in **only one**.

```
maths = {"Aman", "Riya", "Kabir"}
science = {"Riya", "Kabir", "Meera"}

print("Both:", sorted(maths & science))
print("Either:", sorted(maths | science))
print("Only maths:", sorted(maths - science))
print("Only one:", sorted(maths ^ science))
```

OUTPUT

```
Both: ['Kabir', 'Riya']
Either: ['Aman', 'Kabir', 'Meera', 'Riya']
Only maths: ['Aman']
Only one: ['Aman', 'Meera']
```

Unique letters

```
word = "mississippi"
print(len(word))
print(len(set(word)))
print(sorted(set(word)))
```

OUTPUT

```
11
4
['i', 'm', 'p', 's']
```

Common items of two lists

```
a = [1, 2, 3, 3]
b = [3, 4, 5]
print(sorted(set(a) & set(b)))
```

OUTPUT

```
[3]
```

List, tuple or set?

	LIST	TUPLE	SET
Brackets	`[ ]`	`( )`	`{ }`
Ordered	**Yes**	**Yes**	**No**
Duplicates	allowed	allowed	**removed**
Can change	**Yes**	**No**	**Yes**
Indexing	`x[0]`	`x[0]`	**no**
Empty one	`[]`	`()`	`set()`

- Need an **ordered, changing** collection? Use a **list**.
- Need **fixed** data that must not change? Use a **tuple**.
- Need **unique** items or fast membership checks and maths between groups? Use a **set**.

Ask

Which one would you pick for: the months of the year, the unique visitors to a site, a shopping cart?

Practice

Warm-up questions

1. Make a tuple of 5 numbers and print its length, first and last item.
2. Unpack `(10, 20, 30)` into three variables and print their sum.
3. Swap two variables without a third variable.
4. Print the unique items of the list `[5, 3, 5, 1, 3]`.
5. Print the common items of two sets.

Harder questions

6. Write a function that returns the minimum, maximum and average of a list as a tuple.

7. Find the items that are in the first list but not in the second.
8. Check whether a string has **all unique** characters.
9. Count how many different words are in a sentence.
10. Given `(1, 2, 3, 4, 5)`, use `*` to print the first, the last and the middle items.

Find the output

Work out the output first, then press Run to check

```
t = (1, 2, 3, 4, 5)
a, *b, c = t
print(a, b, c)
s = {1, 2, 3}
s.add(3)
s.add(4)
print(len(s))
print({1, 2, 3} & {2, 3, 4})
print({1, 2, 3} - {2, 3, 4})
```

Try the unique-word question

Your turn: count the different words, then press Run

```
sentence = "the cat and the dog and the bird"
# print how many different words there are
```

Homework and wrap-up

- Write a program that stores a **student record** `(name, age, marks)` as a tuple and prints it with **unpacking**.
- Write a program that removes the **duplicates** from a list and keeps the **original order**.
- Write a program that finds the **union, intersection and difference** of two sets typed by the user.
- Write a program that checks whether two strings are **anagrams** using `sorted()`.
- Revise the **list, tuple and set** comparison table.

Next class

Dictionaries.